Dynamic Programming

1) Solve the problem recursively.
2) Observe the recursion tree and find out the answer the building.
3) Memoize the overlapping subproblems.
4) Change it to tabulation.
5) Optimize more if you can.

2 min

Observe how memo array is being build

Inside for loop
- replace return → continue
- rec. calls → dp array

Observe where your answer will lie.

Ques Find n^{th} term of Fib series

n = 5
n = 7 → 13

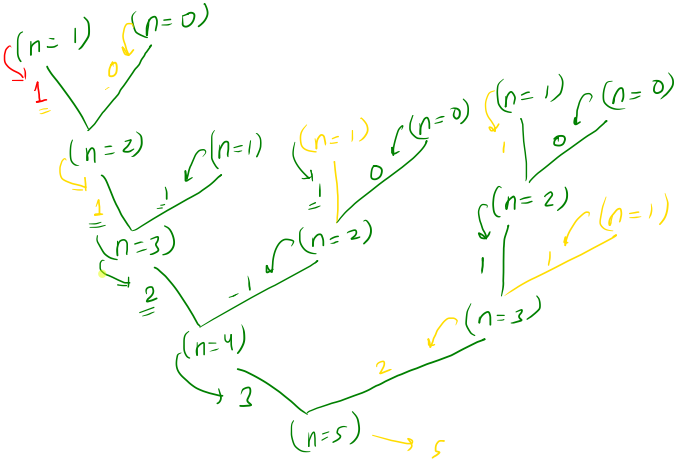
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55

$fib(n) = fib(n-1) + fib(n-2)$;

```
public static int fib(int n){
  1) if(n <= 1){
        return n;
     }

  2) int lastTerm = fib(n-1);
  3) int secondLastTerm = fib(n-2);

  4) int nthTerm = lastTerm + secondLastTerm;
     return nthTerm;
}
```

Array → indices arr(i) = x

2^n



```java
public static int fib_memo(int n, int[] memo){
    if(n <= 1){
        memo[n] = n;
        return n;
    }

    if(memo[n] != -1){
        return memo[n];
    }

    int lastTerm = fib_memo(n-1, memo);
    int secondLastTerm = fib_memo(n-2, memo);

    int nthTerm = lastTerm + secondLastTerm;

    memo[n] = nthTerm;
    return nthTerm;
}

public static void main(String[] args) {
    int n = 5;
    int[] memo = new int[n+1];
    Arrays.fill(memo, -1);

    System.out.println(fib_memo(n, memo));
}
```

memo(n) == -1 → ans not calculated yet

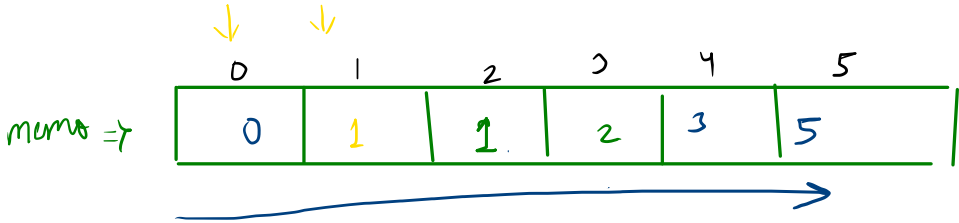
1, 0

2

3

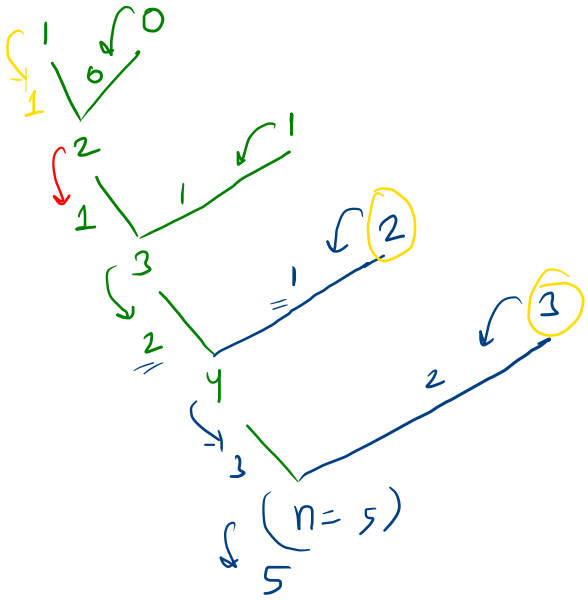
4

5



$fib(n) = fib(n-2) + fib(n-1)$;

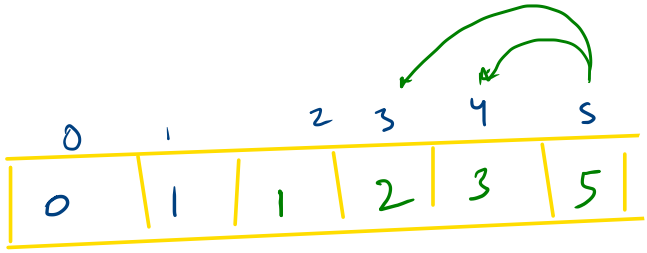
$memo(n) = memo[n-2] + memo[n-1]$;



```java
public static int fib_Tab(int N, int[] memo){
    for(int n=0; n<=N; n++){
        if(n <= 1){
            memo[n] = n;
            continue;
        }

        int lastTerm = memo[n-1]; //fib_memo(n-1, memo);
        int secondLastTerm = memo[n-2]; //fib_memo(n-2, memo);

        int nthTerm = lastTerm + secondLastTerm;

        memo[n] = nthTerm;
    }
}
```

5

2

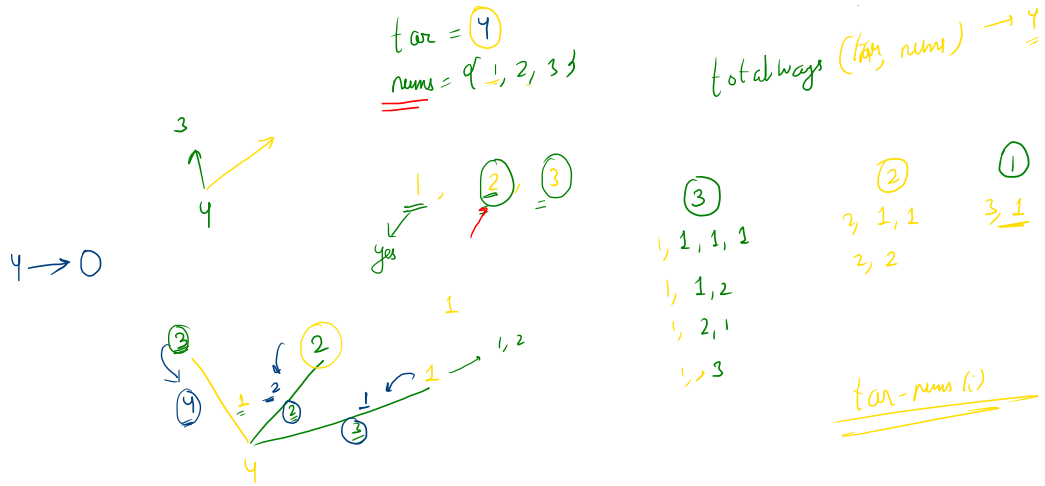


n = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4

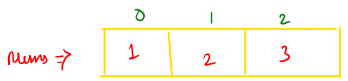
lt = 1

slt = 0

leet 377

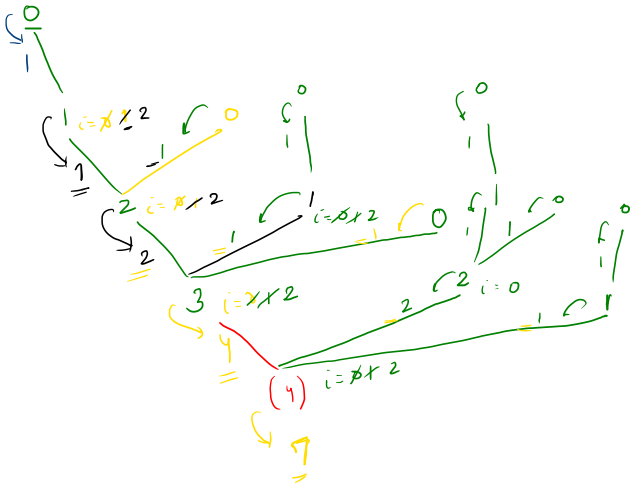



```java
public int rec(int[] nums, int target){
    if(target == 0){
        return 1;
    }

    int totalWays = 0;
    for(int i=0; i<nums.length; i++){
        if(target - nums[i] >= 0){
            totalWays += rec(nums, target - nums[i]);
        }
    }

    return totalWays;
}

public int combinationSum4(int[] nums, int target) {
    return rec(nums,target);
}
```

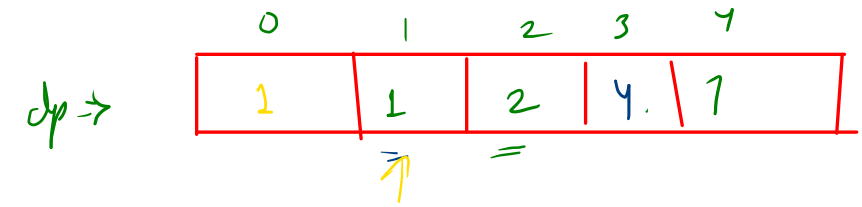


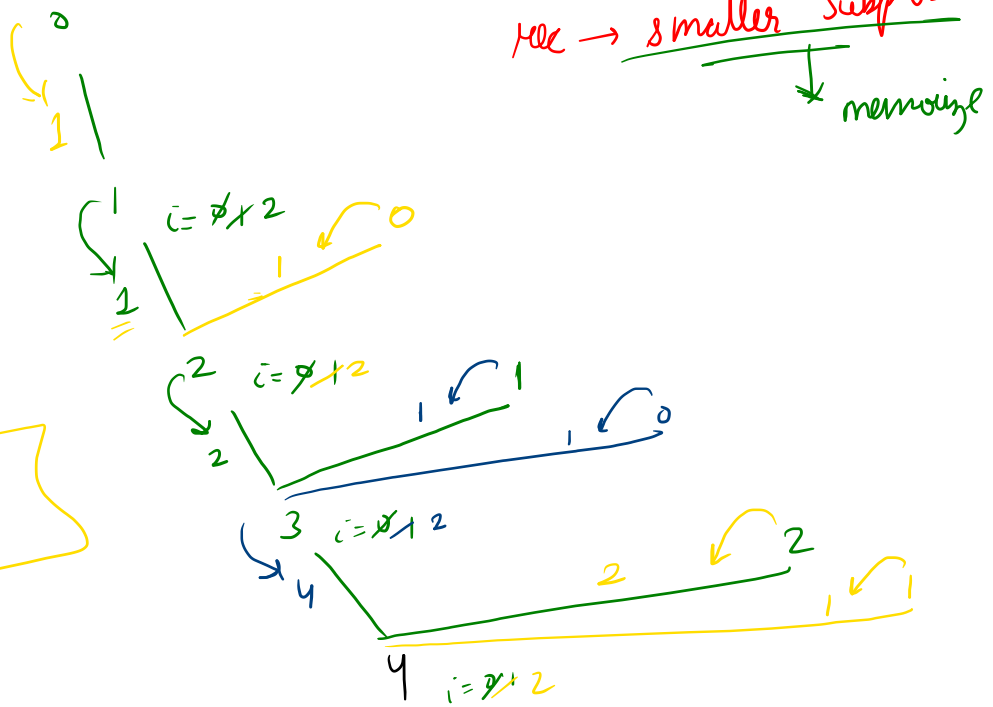
(nums.length) target

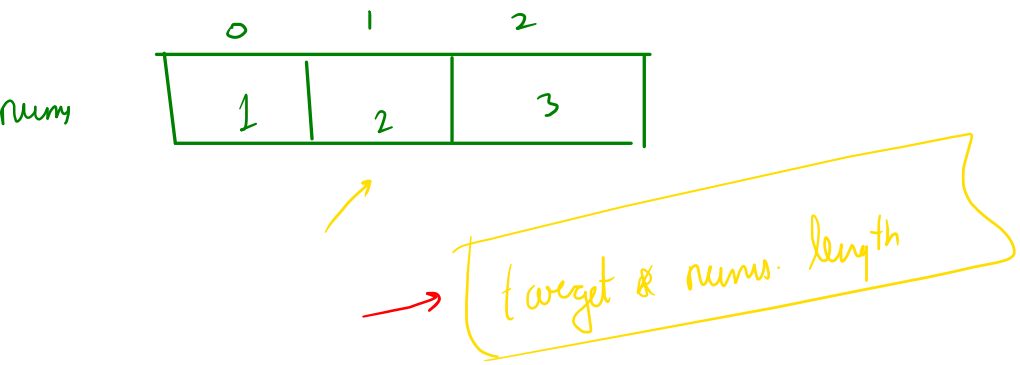
(number of calls) ∩ height of tree



```java
public int rec_memo(int[] nums, int target, int[] dp){
    if(target == 0){
        return dp[target] = 1;
    }

    if(dp[target] != -1){
        return dp[target];
    }

    int totalWays = 0;
    for(int i=0; i<nums.length; i++){
        if(target - nums[i] >= 0){
            totalWays += rec_memo(nums, target - nums[i], dp);
        }
    }

    return dp[target] = totalWays;
}
```

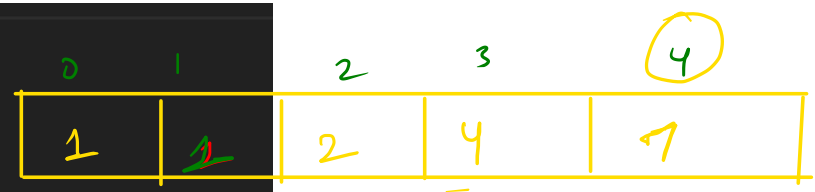






```java
public int rec_tab(int[] nums, int Target, int[] dp){
    for(int target=0; target<=Target; target++){
        if(target == 0){
            dp[target] = 1;
            continue;
        }

        int totalWays = 0;
        for(int i=0; i<nums.length; i++){
            if(target - nums[i] >= 0){
                totalWays += dp[target-nums[i]]; //rec_memo(nums, target - nums[i], dp);
            }
        }

        dp[target] = totalWays;
    }

    return dp[Target];
}
```

no target



target = 0 + 1 + 2 = 3 4

target - nums[i]

4 - 3 ≥ 0

$dp(4) = dp(4-1) + dp(4-2) + dp(4-3)$

$dp(3) + dp(2) + dp(1)$

4 2 1

tar = 3 4

$dp(3) = dp(2) + dp(1) + dp(0)$

2 1 1

Target sum subset

subset

```
N = 6
arr[] = {3, 34, 4, 12, 5, 2}
sum = 9
```

target

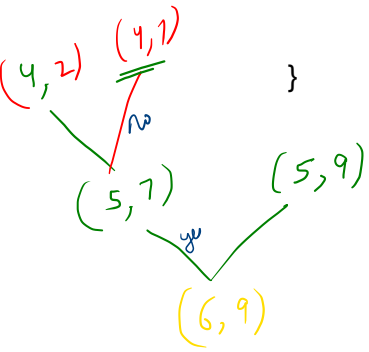
arr ⇒ {1, 2, 3}

yes no

no no no {}
no no yes {3}
no yes no {2}
yes no no {1}
yes no yes {1,3}
yes yes no {1,2}
yes yes yes {1,2,3}
no yes yes {2,3}

pro-active why?

```
public static boolean targetSum(int[] arr, int N, int target){

}
```



6 elements → 9

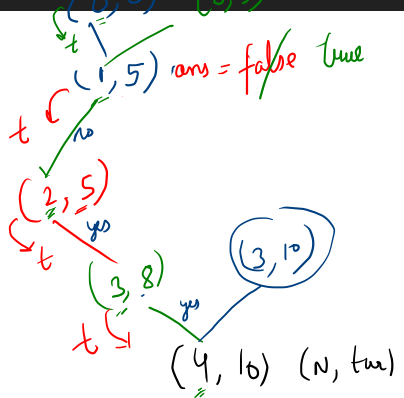
```
N = 6
arr[] = {3, 34, 4, 12, 5, 2}
sum = 9
```

```
public static Boolean targetSumSubsetRec(int N, int target, int[] arr){
    if(N==0){
        return target == 0;
    }

    boolean ans = false;

    // yes call
    if(target - arr[N-1] >= 0){
        ans = targetSumSubsetRec(N-1, target - arr[N-1], arr);
    }

    // no call
    ans = ans || targetSumSubsetRec(N-1, target, arr);

    return ans;
}
```

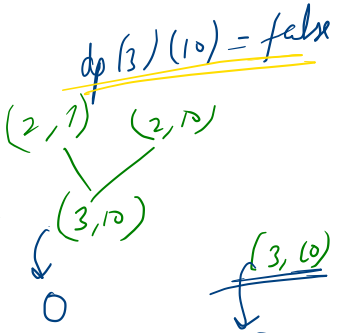
arr = {5, 8, 3, 2} → 2

target = 10

Overlapping subproblems



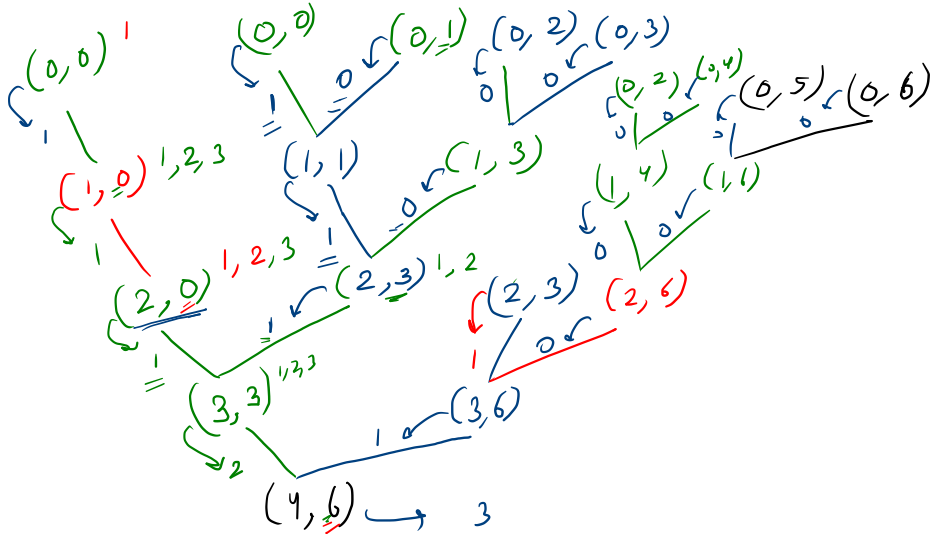
subset



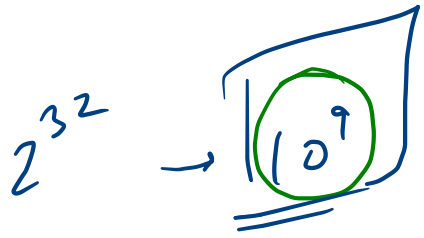
Number of Subsets with sum = target.

arr = {1, 2, 3, 3} tar = 6

number of subsets using 4 elements with sum = 6




```java
// number of subsets with sum == target
public static int totalSubsets_rec(int[] arr, int N, int tar){
    if(N==0){
        return tar == 0 ? 1 : 0;
    }

    int ways = 0;
    // yes call
    if(tar - arr[N-1] >= 0){
        ways += totalSubsets_rec(arr,N-1,tar-arr[N-1]);
    }

    // no call
    ways += totalSubsets_rec(arr,N-1, tar);

    return ways;
}
```



$N = 10000000000$

$N = 10^5$

↳ $a \log n$ $(aN$

10^{18}

$O(1)$

$O(N)$

$O(N)$

$\log N$